Tina4 Ruby ORM

🔥 **Hot Tips**

- You **never** need the ORM — raw `Database` is perfect for 90% of cases
- But when you want it... it feels like writing plain Ruby classes
- Zero boilerplate — field types are declared with a clean DSL
- Works with SQLite, PostgreSQL, MySQL, MSSQL, Firebird — all the same

Quick Start

```ruby
require "tina4"

class User < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field  :name
  string_field  :email
  datetime_field :created_at
end

# That's it — use the global database from Tina4.database
user = User.create(name: "Alice", email: "alice@example.com")
puts user.id          # → 1

user.name = "Alice Wonder"
user.save             # → UPDATE
```

```ruby
user = User.find(1)
puts user.name        # → "Alice Wonder"
```

Field Types

Field Method	Ruby type	SQL example	Notes
`integer_field`	`Integer`	`INTEGER`	
`string_field`	`String`	`VARCHAR(255)`	`length: 500` optional
`text_field`	`String`	`TEXT`	unlimited
`float_field`	`Float`	`FLOAT`	
`decimal_field`	`BigDecimal`	`NUMERIC(10,2)`	`precision: , scale:`
`boolean_field`	`Boolean`	`BOOLEAN`	
`date_field`	`Date`	`DATE`	
`datetime_field`	`DateTime`	`TIMESTAMP`	
`timestamp_field`	`DateTime`	`TIMESTAMP`	alias
`blob_field`	`String`	`BLOB`	binary
`json_field`	`Hash/Array`	`JSON`	

Field Options

ruby

```ruby
class Product < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field  :name, length: 100, nullable: false
  decimal_field :price, precision: 10, scale: 2, default: 0.0
  boolean_field :active, default: true
  datetime_field :created_at
end
```

Custom Table Name

ruby

```ruby
class User < Tina4::ORM
  table_name "people"     # Override default "users"
  integer_field :id, primary_key: true, auto_increment: true
  string_field :name
end
```

Core Methods

Method	Example	What it does	Returns
`.find(id)`	`User.find(1)`	Load by primary key	Instance or `nil`
`.where(cond, params)`	`User.where("age > ?", [30])`	Query with conditions	`Array` of instances
`.all(limit:, skip:, order_by:)`	`User.all(limit: 10)`	All records	`Array` of instances
`.count(cond, params)`	`User.count("active = ?", [true])`	Count records	`Integer`
`.create(attrs)`	`User.create(name: "Bob")`	Insert and return	Instance
`#save`	`user.save`	INSERT or UPDATE	`true/false`
`#delete`	`user.delete`	Delete record	`true/false`
`#load(id)`	`user.load(1)`	Load into instance	`true/false`
`#to_hash`	`user.to_hash`	Convert to hash	`Hash`

Method	Example	What it does	Returns
`#to_json`	`user.to_json`	Convert to JSON	`String`
`#persisted?`	`user.persisted?`	Is saved to DB?	`Boolean`
`#errors`	`user.errors`	Validation errors	`Array`

Full Example

ruby

```ruby
require "tina4"

class Category < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :name
end

class Article < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :title
  text_field :content
  integer_field :category_id
  datetime_field :created_at
end

# Use it
cat = Category.create(name: "Tech")

article = Article.create(
  title: "Tina4 is awesome",
  content: "Great framework",
  category_id: cat.id
)

# Query
```

```
articles = Article.where("category_id = ?", [cat.id])
articles.each { |a| puts a.title }
```

Using with Routes

ruby

```ruby
Tina4.get "/api/users" do |request, response|
  users = User.all(limit: 20)
  response.json(users.map(&:to_hash))
end

Tina4.get "/api/users/{id:int}" do |request, response|
  user = User.find(request.params["id"])
  user ? response.json(user.to_hash) : response.json({ error: "Not found" }, 40
end

Tina4.post "/api/users" do |request, response|
  user = User.create(request.json_body)
  if user.persisted?
    response.json(user.to_hash, 201)
  else
    response.json({ errors: user.errors }, 422)
  end
end
```

Summary – The Tina4 ORM Philosophy

- **No migrations required** for ORM classes
- **No query builder hell**
- **No session management**
- **No associations to configure**

Just:

ruby

```ruby
user = User.create(name: "Bob")

Tina4.post "/api/users" do |request, response|
  user = User.create(request.json_body)
  response.json(user.to_hash, 201)
end
```

Previous page
Migrations

Next page
CRUD